Projektdokumentation

Aufbau eines dedizierten OpenArena-Spieleservers auf einer Microsoft-Azure-VM mit Docker

Privates Infrastruktur-/Hobbyprojekt

Plattform: Microsoft Azure (Free Tier) • Ubuntu 24.04 LTS • Docker

Durchführungszeitraum: Juni 2026

Stand: 18. Juni 2026

Inhaltsverzeichnis

Inhaltsverzeichnis....2
1 Einleitung....3
2 Projektbeschreibung....3
2.1 Projektumfeld....3
2.2 Ziel des Projektes....3
2.3 Projektabgrenzung....3
3 Projektplanung....3
3.1 Ist-Analyse / Ausgangslage....3
3.2 Soll-Konzept (Architektur)....4
3.3 Technologieentscheidung: Quake III Arena vs. OpenArena....4
4 Realisierung....5
4.1 Einrichtung der Azure-Umgebung....5
4.2 Erster Lösungsansatz: Quake III Arena (verworfen)....6
4.3 Strategiewechsel: Recherche und Entscheidung für OpenArena....7
4.4 Aufbau des finalen Docker-Setups....7
Dockerfile....7
server.cfg....8
docker-compose.yml....8
4.5 Build- und Compose-Probleme lösen....9
4.6 Serverstart und Verifikation....9
4.7 Netzwerkfreigabe und Verbindungstest....9
5 Qualitätssicherung....10
6 Projektabschluss....10
6.1 Soll-Ist-Vergleich....10
6.2 Fazit und Ausblick....10
Anhang....11
A.1 Befehlsübersicht....11
A.2 Glossar....11
A.3 Quellen....11

1 Einleitung

Diese Dokumentation beschreibt die eigenständige Planung, Umsetzung und Inbetriebnahme eines dedizierten Multiplayer-Spieleservers auf Basis von OpenArena/ioquake3, betrieben in einem Docker-Container auf einer virtuellen Maschine in der Microsoft-Azure-Cloud.

Im Verlauf des Projekts wurde zunächst ein anderer Lösungsweg verfolgt, der sich aus mehreren Gründen als nicht praktikabel erwies. Die Dokumentation zeigt daher bewusst sowohl die anfänglichen Probleme als auch den Strategiewechsel, der letztlich zum erfolgreichen Projektabschluss geführt hat.

Bei der Recherche und Fehlerbehebung wurden unterstützend KI-Assistenten genutzt; alle Konfigurationsentscheidungen und Tests wurden eigenständig auf dem Server durchgeführt und verifiziert.

2 Projektbeschreibung

2.1 Projektumfeld

Das Projekt wurde im privaten Umfeld als Hobby-/Lab-Projekt durchgeführt, mit dem Ziel, praktische Erfahrung mit Cloud-Infrastruktur (Microsoft Azure), Containerisierung (Docker) und Linux-Serveradministration zu sammeln. Genutzt wurde ein kostenloses Azure-Free-Tier-Konto.

2.2 Ziel des Projektes

Ziel war die Inbetriebnahme eines dauerhaft erreichbaren, dedizierten Servers für einen Open-Source-Mehrspieler-Shooter, sodass sich mehrere Spieler unabhängig von Ort und eigener Hardware über das Internet verbinden können. Der Server sollte containerisiert betrieben werden, um Wartung, Neustarts und spätere Anpassungen zu vereinfachen.

2.3 Projektabgrenzung

Folgende Punkte waren nicht Bestandteil des Projekts:

- Eigener Build der Spiel-Engine aus dem Quellcode (zugunsten eines fertigen, offiziell gepflegten Pakets)
- Einbindung zusätzlicher Maps, Mods oder eines Voice-Chat-Systems
- Automatisiertes Monitoring, Backups oder Skalierung über mehrere VMs
- Absicherung über ein eigenes VPN; der Zugriff erfolgt direkt über die öffentliche IP-Adresse der VM

3 Projektplanung

3.1 Ist-Analyse / Ausgangslage

Vor Projektbeginn existierte keine eigene Server- oder Cloud-Infrastruktur. Bekannt war lediglich, dass die Engine „ioquake3“ quelloffen (GPL) ist, die Original-Spieldateien von Quake III Arena (insbesondere „pak0.pk3“) jedoch weiterhin urheberrechtlich geschützt sind und nicht frei verteilt werden dürfen. Diese Einschränkung war zu Projektbeginn nicht vollständig bekannt und führte, wie in Kapitel 4 beschrieben, zu mehreren Fehlversuchen.

3.2 Soll-Konzept (Architektur)

Geplant wurde folgende Architektur: Eine Azure-VM mit Ubuntu-Server stellt über Docker einen Container bereit, der die Spiel-Engine inklusive freier Spieldaten enthält. Der Container ist über einen gemounteten Ordner mit einer eigenen Konfigurationsdatei (server.cfg) versehen. Eine Azure-Netzwerksicherheitsgruppe (NSG) gibt den benötigten UDP-Port für eingehende Verbindungen frei, sodass sich Clients von außen verbinden können.

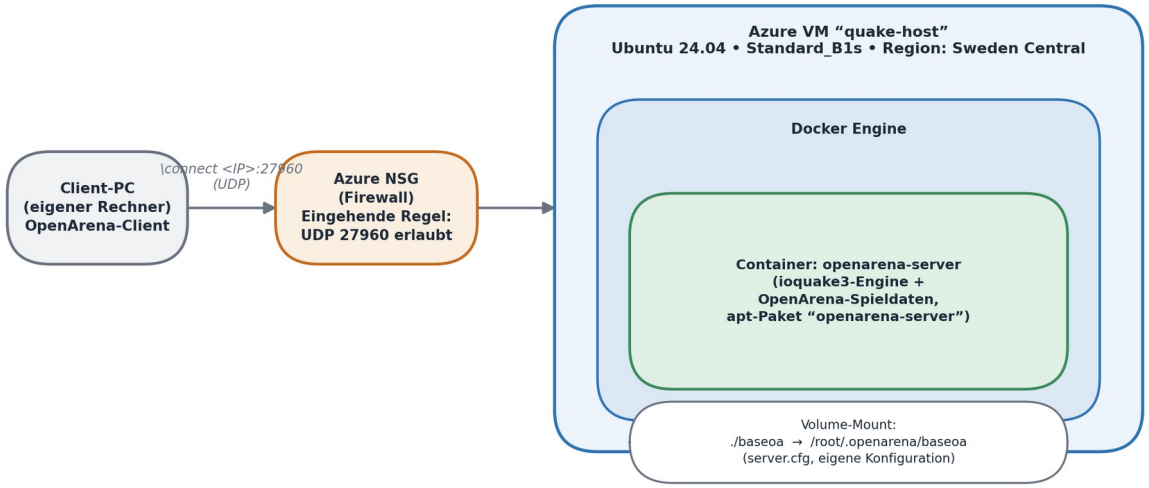


Abbildung 1: Geplante Architektur – Client, Azure-Netzwerk und Docker-Container im Zusammenspiel

3.3 Technologieentscheidung: Quake III Arena vs. OpenArena

Im Projektverlauf stellte sich die Frage, ob der Server auf Basis des kommerziellen Quake III Arena (mit der ioquake3-Engine) oder auf Basis des freien OpenArena-Projekts (ebenfalls ioquake3-Engine, aber vollständig freie Spieldaten) aufgebaut werden soll. Die folgende Tabelle fasst die Entscheidung zusammen, die letztlich zum in Kapitel 4 beschriebenen Strategiewechsel führte:

Kriterium	Quake III Arena (ioquake3)	OpenArena
Spieldaten frei verfügbar	Nein – „pak0.pk3“ erfordert gekaufte Vollversion	Ja – vollständig frei (GPL/CC)
Setup-Aufwand	Hoch – manueller Engine-Build, Asset-Beschaffung nötig	Niedrig – fertiges, offizielles apt-Paket
Wartung/Updates	Manuell	Über den Paketmanager (apt)
Bot-/Multiplayer-Unterstützung	Ja	Ja

Auf Basis dieser Gegenüberstellung wurde OpenArena gewählt: Es nutzt dieselbe Engine, ist aber rechtlich unbedenklich und lässt sich über ein offizielles Ubuntu-Paket deutlich robuster betreiben.

4 Realisierung

Die folgende Abbildung gibt einen Gesamtüberblick über den tatsächlichen Projektverlauf – inklusive der beiden Hauptprobleme, die zwischenzeitlich aufgetreten sind und gelöst werden mussten. Die einzelnen Schritte werden anschließend im Detail beschrieben.

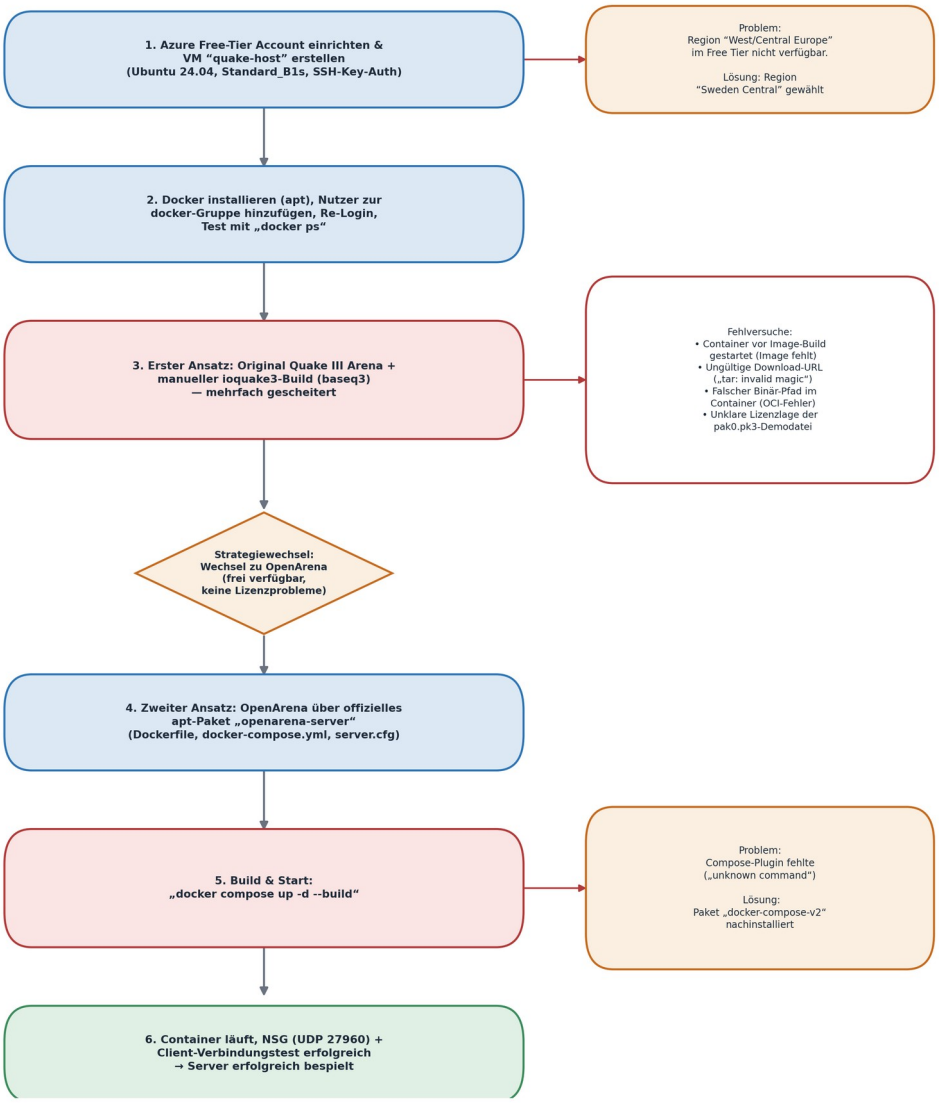


Abbildung 2: Tatsächlicher Projektablauf inklusive aufgetretener Probleme und Lösungen

4.1 Einrichtung der Azure-Umgebung

Im Azure-Portal wurde mit dem kostenlosen Free-Tier-Konto eine neue virtuelle Maschine erstellt. Folgende Konfiguration wurde gewählt:

- Image: Ubuntu Server 24.04 LTS (reguläres Ubuntu, keine Minimal- oder Alpine-Variante, da Alpine auf Azure-Hostebene Erfahrungsgemäß Probleme mit den Netzwerktreibern verursachen kann)
- Größe: Standard_B1s (1 vCPU, 1 GB RAM) – die im Free Tier 12 Monate kostenlose Instanzgröße
- Authentifizierung: SSH-öffentlicher Schlüssel (Azure generiert ein Schlüsselpaar zum Download als .pem-Datei)
- Netzwerk: vorbereitende Regel für eingehenden UDP-Verkehr auf Port 27960 in der Netzwerksicherheitsgruppe (NSG)

Bei der Regionswahl zeigte sich ein erstes, kleines Problem: Die Region „West/Central Europe“ stand für das Free-Tier-Kontingent nicht zur Verfügung. Nach kurzem Ausprobieren verschiedener Regionen erwies sich „Sweden Central“ als sinnvollste verfügbare Alternative (gute Anbindung, Free-Tier-fähig) und wurde als Standort gewählt.

Die SSH-Verbindung zur VM wurde über den heruntergeladenen privaten Schlüssel aufgebaut. Da SSH Verbindungen mit zu offenen Dateirechten aus Sicherheitsgründen verweigert, mussten die Rechte der Schlüsseldatei zuvor angepasst werden:

```
chmod 400 quake-schluessel.pem
ssh -i quake-schluessel.pem azureuser@<Azure-IP>
```

Anschließend wurde Docker auf der VM installiert und der eigene Nutzer der docker-Gruppe hinzugefügt, damit Docker-Befehle ohne „sudo“ ausgeführt werden können:

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
exit   # Sitzung trennen, damit die Gruppenrechte greifen
ssh -i quake-schluessel.pem azureuser@<Azure-IP>   # erneut verbinden
docker ps   # Funktionstest
```

Der erfolgreiche Funktionstest mit „docker ps“ bestätigte, dass Docker korrekt installiert war und ohne erhöhte Rechte genutzt werden konnte.

4.2 Erster Lösungsansatz: Quake III Arena (verworfen)

Der ursprüngliche Plan war, die Original-Spieldateien von Quake III Arena zusammen mit einem selbst gebauten ioquake3-Docker-Image zu verwenden. Hierfür wurde zunächst ein Alpine-basiertes Image erstellt, das die ioquake3-Server-Binärdatei per „curl“ aus dem Internet nachladen sollte.

Bereits beim ersten Startversuch zeigte sich ein einfacher, aber lehrreicher Fehler: Der Container wurde gestartet, bevor das zugehörige Image überhaupt gebaut worden war.

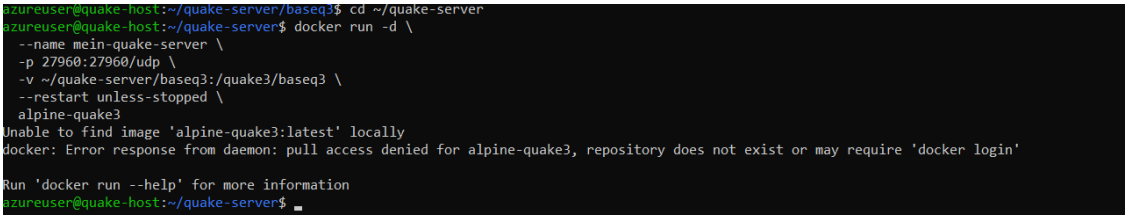


Abbildung 3: „docker run“ schlägt fehl, da das referenzierte Image „alpine-quake3“ noch nicht existiert

Nach dem Anlegen des Dockerfiles und einem erfolgreichen Start des Build-Vorgangs brach der Build an einer anderen Stelle ab: Die per „curl“ heruntergeladene Datei war keine gültige tar.gz-Archivdatei (vermutlich eine veraltete oder nicht mehr existierende Release-URL), wodurch das Entpacken fehlschlug:

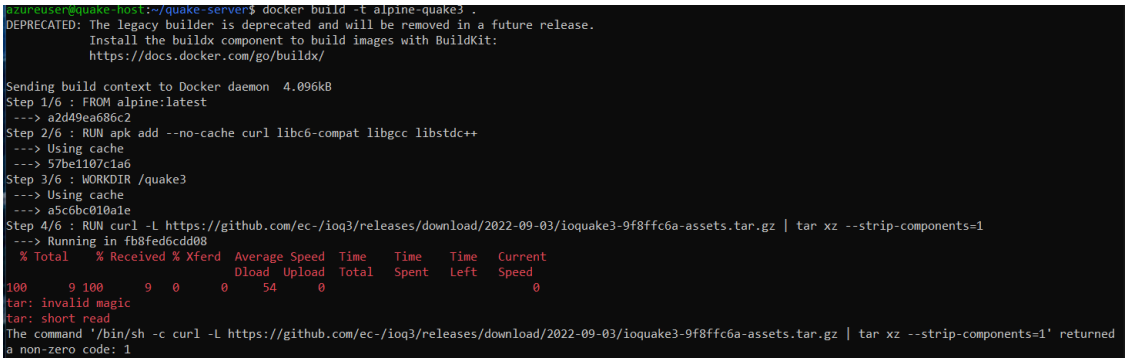


Abbildung 4: Der Build bricht ab, da die heruntergeladene Datei kein gültiges tar.gz-Archiv ist („tar: invalid magic“)

Als Notlösung wurde versucht, eine frei verfügbare Demo-Version der Datei „pak0.pk3“ aus einem Drittanbieter-Repository nachzuladen und einen alternativen, auf Ubuntu basierenden Container zu starten. Auch dieser Versuch scheiterte: Der im Dockerfile angegebene Pfad zur Server-Binärdatei existierte im Container nicht, sodass die Laufzeitumgebung (OCI-Runtime) den Start verweigerte:

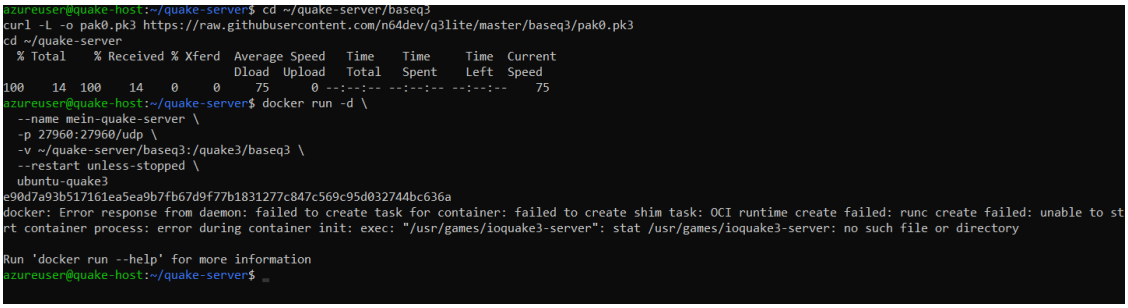


Abbildung 5: Der Container-Start schlägt fehl, da die angegebene Server-Binärdatei im Image nicht existiert

Diese Häufung von Problemen – instabile/veraltete Download-Quellen, ein falsch angenommener Dateipfad und vor allem die unklare rechtliche Situation rund um die Demo-pak0.pk3 – führte zu der Entscheidung, diesen Ansatz nicht weiter zu verfolgen.

4.3 Strategiewechsel: Recherche und Entscheidung für OpenArena

Als Grundlage für die weitere Vorgehensweise wurde der offizielle „ioquake3 Sys Admin Guide“ (ioquake3.org) konsultiert. Daraus ging hervor, dass die Engine grundsätzlich unabhängig von den verwendeten Spieldaten ist („fs_basegame“ bzw. „fs_game“) und dass dieselbe Engine genauso gut mit dem vollständig freien Spiel OpenArena betrieben werden kann.

Daraufhin wurde die Entscheidung getroffen, auf OpenArena zu wechseln (siehe Entscheidungsmatrix in Kapitel 3.3). Statt die Engine erneut manuell zu bauen, wurde zudem das offizielle Ubuntu-Paket „openarena-server“ verwendet, das die ioquake3-Engine und alle benötigten Spieldaten in geprüfter, stabiler Form bereitstellt – wodurch die zuvor aufgetretenen Probleme mit instabilen Download-Quellen entfielen.

4.4 Aufbau des finalen Docker-Setups

Das finale Setup besteht aus drei Dateien: einem Dockerfile, das Ubuntu 24.04 als Basis nutzt und das Paket „openarena-server“ installiert, einer „server.cfg“ für die individuelle Serverkonfiguration sowie einer „docker-compose.yml“ zur Verwaltung von Build, Start und Volume-Mount.

Dockerfile

```
FROM ubuntu:24.04
```

```
ENV DEBIAN_FRONTEND=noninteractive

# „universe“-Repository sicherstellen (enthält openarena-server)
RUN apt-get update && apt-get install -y --no-install-recommends software-
properties-common \
    && add-apt-repository -y universe \
    && apt-get update

# OpenArena Dedicated Server inkl. ioquake3-Engine und baseoa-Spieldaten
RUN apt-get install -y openarena-server \
    && apt-get purge -y software-properties-common \
    && apt-get autoremove -y \
    && rm -rf /var/lib/apt/lists/*

RUN mkdir -p /root/.openarena/baseoa

EXPOSE 27960/udp

ENTRYPOINT ["/usr/games/openarena-server"]
CMD ["+set", "dedicated", "2", "+set", "net_port", "27960", "+exec",
"server.cfg"]
```

server.cfg

```
seta sv_hostname "Mein OpenArena Server"
seta sv_maxclients 8
seta g_gametype 0
seta sv_pure 1
seta com_hunkmegs 128
seta rconpassword "EinSicheresEinzigartigesPasswort"

seta bot_enable 1
seta bot_minplayers 2

map oa_dm4
```

docker-compose.yml

```
services:
  openarena:
    build: .
    container_name: openarena-server
    restart: unless-stopped
    ports:
      - "27960:27960/udp"
    volumes:
      - ./baseoa:/root/.openarena/baseoa
```

4.5 Build- und Compose-Probleme lösen

Beim ersten Start mit „docker compose up -d --build“ zeigte sich, dass das Compose-Plugin auf der VM gar nicht installiert war („docker: unknown command: docker compose“). Der naheliegende Lösungsversuch über das Paket „docker-compose-plugin“ schlug fehl, da dieses Paket in den Standard-Ubuntu-Repositories unter diesem Namen nicht existiert, wenn Docker zuvor über das einfache Paket „docker.io“ installiert wurde:

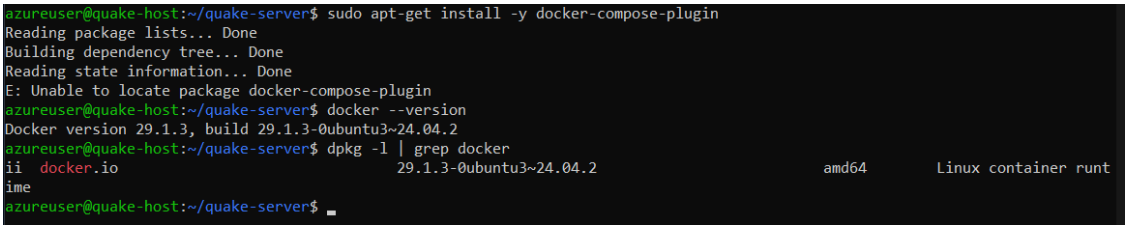


Abbildung 6: Das Paket „docker-compose-plugin“ ist unter Ubuntu mit „docker.io“ nicht verfügbar

Die Lösung war die Installation des alternativen, in den Ubuntu-Repositories vorhandenen Pakets:

```
sudo apt-get install -y docker-compose-v2
docker compose version
```

Damit stand das Compose-Plugin zur Verfügung und der eigentliche Build-Befehl konnte erneut ausgeführt werden.

4.6 Serverstart und Verifikation

Mit installiertem Compose-Plugin konnte das Image gebaut und der Container gestartet werden:

```
cd ~/quake-server
docker compose up -d --build
docker compose logs -f
```

Die Live-Logs zeigten den erfolgreichen Start des OpenArena-Servers, das Laden der Standard-Map sowie das Lauschen auf dem konfigurierten UDP-Port. Der Container lief stabil im Hintergrund weiter, auch nachdem die Log-Ansicht im Terminal verlassen wurde.

4.7 Netzwerkfreigabe und Verbindungstest

Die Azure-NSG-Regel für UDP-Port 27960 war bereits in der Vorbereitungsphase (Kapitel 4.1) angelegt worden. Zusätzlich wurde auf der VM geprüft, ob eine lokale Firewall (ufw) aktiv ist und den Port gegebenenfalls blockiert:

```
sudo ufw status
sudo ufw allow 27960/udp   # falls ufw aktiv ist
```

Auf dem eigenen Rechner wurde anschließend ein OpenArena-Client installiert und über die Konsole eine Verbindung zur öffentlichen IP-Adresse der Azure-VM aufgebaut:

```
\connect <öffentliche-IP-der-VM>:27960
```

Die Verbindung wurde erfolgreich aufgebaut; der Server konnte live bespielt werden. Damit war der praktische Funktionsnachweis des Projekts erbracht.

5 Qualitätssicherung

Vor dem Projektabschluss wurden folgende Prüfungen durchgeführt, um den ordnungsgemäßen Betrieb des Servers sicherzustellen:

- Container-Status: „docker compose ps“ zeigt den Container „openarena-server“ mit Status „Up“
- Log-Kontrolle: „docker compose logs“ bestätigt den erfolgreichen Start der Engine, das Laden der Map „oa_dm4“ und das Lauschen auf UDP-Port 27960
- Persistenz: Die Server-Konfiguration (server.cfg) bleibt durch den Volume-Mount auch nach einem Neubau des Containers erhalten
- Neustart-Verhalten: „restart: unless-stopped“ sorgt dafür, dass der Container nach einem VM-Neustart automatisch wieder hochfährt
- Netzwerkverbindung: Verbindungsaufbau von einem externen OpenArena-Client über die öffentliche IP-Adresse der VM war erfolgreich
- Spielbarkeit: Der Server wurde live bespielt; Bewegungen, Schüsse und Spielablauf funktionierten ohne erkennbare Latenz- oder Stabilitätsprobleme

Damit wurden sowohl die technische Funktion (Container läuft stabil) als auch der eigentliche Anwendungsfall (Server ist von außen erreichbar und spielbar) erfolgreich verifiziert.

6 Projektabschluss

6.1 Soll-Ist-Vergleich

Der ursprüngliche Plan, einen Server auf Basis von Quake III Arena mit manuell gebauter ioquake3-Engine zu betreiben, musste aufgegeben werden. Anstelle dessen wurde – wie in Kapitel 3.3 begründet – auf OpenArena mit einem offiziellen, vorgefertigten apt-Paket umgeschwenkt. Das eigentliche Projektziel, ein dauerhaft erreichbarer, containerisierter Multiplayer-Server, wurde dadurch nicht geändert und letztlich vollständig erreicht.

Zusätzlich zum eigentlichen Server-Setup musste unterwegs ein nicht eingeplantes Teilproblem gelöst werden: das fehlende Docker-Compose-Plugin auf der VM. Dies verursachte einen kleinen Mehraufwand, hatte aber keinen Einfluss auf das Endergebnis.

6.2 Fazit und Ausblick

Das Projekt wurde erfolgreich abgeschlossen: Der OpenArena-Server läuft stabil in einem Docker-Container auf einer Azure-Free-Tier-VM und ist über das Internet erreichbar und spielbar.

Die wichtigste Erkenntnis aus dem Projekt betrifft die Wahl der Software-Quelle: Der manuelle Bezug von Engine-Binärdateien und Spieldaten über einzelne, nicht offiziell gepflegte Download-Links erwies sich als fehleranfällig und zudem rechtlich heikel. Ein offiziell gepflegtes Distributionspaket (hier: „openarena-server“ über apt) war hingegen sowohl stabiler als auch unkomplizierter in der Einrichtung. Ebenso zeigte sich, dass bei einer schlanken Docker-Installation über „docker.io“ das Compose-Plugin separat nachinstalliert werden muss.

Als mögliche nächste Schritte für eine Weiterentwicklung des Projekts kommen in Frage:

- Einrichtung von HTTP-FastDownload (sv_dlURL), um zusätzliche Maps automatisch an Clients auszuliefern
- Einbindung einfacher Überwachung (z. B. Health-Check oder externe Erreichbarkeitsprüfung des UDP-Ports)
- Automatisiertes Backup der Konfigurationsdateien

- Absicherung des rcon-Zugriffs über ein dediziertes, regelmäßig wechselndes Passwort

Anhang

A.1 Befehlsübersicht

Befehl	Zweck
ssh -i schluessel.pem azureuser@<IP>	SSH-Verbindung zur Azure-VM aufbauen
sudo apt install docker.io -y	Docker Engine installieren
sudo apt-get install -y docker-compose-v2	Docker-Compose-Plugin installieren
docker compose up -d --build	Image bauen und Container im Hintergrund starten
docker compose logs -f	Live-Logs des Containers anzeigen (Beenden: Strg+C, ggf. mehrfach)
docker compose ps	Status der Compose-Services anzeigen
docker compose down	Container stoppen und entfernen
sudo ufw allow 27960/udp	Lokale Firewall (falls aktiv) für UDP 27960 öffnen
\connect <IP>:27960	Im Spiel-Client zum Server verbinden
\rconpassword <Passwort>	Im Spiel-Client bei rcon authentifizieren

A.2 Glossar

Container: Isolierte, leichtgewichtige Laufzeitumgebung für eine Anwendung, verwaltet von Docker

Docker Compose: Werkzeug zur Definition und Verwaltung mehrerer (oder einzelner) Docker-Container über eine YAML-Datei

Dockerfile: Textdatei mit Bauanleitung für ein Docker-Image

ioquake3: Open-Source-Weiterentwicklung der Quake-III-Arena-Engine

NSG: Network Security Group – Azure-Firewall-Komponente auf Netzwerkebene

OpenArena: Vollständig freies Multiplayer-Spiel auf Basis der ioquake3-Engine

rcon: Remote Console – ermöglicht die Fernadministration eines Spieleservers über ein Passwort

UDP: User Datagram Protocol – verbindungsloses Netzwerkprotokoll, von Quake/OpenArena für Spieldaten genutzt

Volume (Docker): Persistenter Speicherbereich, der einen Host-Ordner in einen Container einbindet

A.3 Quellen

- ioquake3 Sys Admin Guide – https://ioquake3.org/help/sys-admin-guide/
- ioquake3-Projekt (GitHub) – https://github.com/ioquake/ioq3
- OpenArena-Projekt – https://www.openarena.ws/

- Microsoft Azure Dokumentation – https://learn.microsoft.com/azure/
- Ubuntu-Paketdatenbank (openarena-server, docker-compose-v2) – https://packages.ubuntu.com/